07 · 高并发网关与任务调度

技术面试准备材料 · AI 应用 / Agent 开发

> 核心问题：Agent 服务的容量怎么算、流量怎么治理、长任务怎么可靠地跑完。

1. 如何估算一个 Agent 服务的容量?

先算单请求的资源消耗，再算系统能承载多少并发。

单请求要看：平均输入/输出 Token、模型单价、平均时延、工具调用次数、外部依赖耗时。然后算单实例在目标利用率下的 QPS。

并发数用 Little's Law：并发 = 到达率 × 平均停留时间。注意并发是结果，不是原因；不能先拍一个「支持 1000 并发」，要看请求停留多久、上游能承受多少。

还要考虑利用率悬崖：CPU 或连接池利用率到 80%～90% 后，排队延迟会非线性上升，所以容量规划要留余量，不能按 100% 算。

Agent 场景的方差很大，平均值不够用，要看 P95/P99。一个长任务可能占用连接几分钟，不能用普通短请求的模型估算。

最后，容量规划要包含降级方案：超过容量时是排队、拒绝还是切小模型，必须在设计时就定好。

2. 为什么 Agent 场景的高并发比普通 API 更难?

有五个原因。

第一，服务时间方差极大。普通 API 可能几十毫秒，Agent 可能几秒到几分钟，长尾请求会占住连接和并发槽。

第二，单请求成本高。一次调用可能几千 Token，重试一次成本翻倍，不能像普通 API 那样随意重试。

第三，请求之间可以合并。相同或相似的问题可以走缓存或 Singleflight，这是普通 API 没有的机会，但也要小心错误合并。

第四，流式响应让连接生命周期变长。一个 SSE 连接可能持续几十秒，网关的连接数、心跳和断线处理都更复杂。

第五，上游是外部依赖，有自己的配额和限流。你不能只控自己的入口，还要控对模型的出口。

所以 Agent 网关不能照搬普通 API 网关，要同时做入口限流、排队、出口配额、缓存和降级。

3. 限流算法怎么选？为什么要分层?

常见三种：计数器/滑动窗口、漏桶、令牌桶。

滑动窗口适合精确控制单位时间内的请求数；漏桶适合平滑输出；令牌桶允许一定突发，最适合 LLM 这种「平时平稳、偶尔爆发」的场景。

分层是因为不同维度都要控：入口按用户/租户限流，防止单租户打满；网关按接口/模型限流，保护上游；上游按厂商配额限流；任务层按并发数限流，避免长任务占满资源。

分布式限流要注意精度：本地令牌桶快但各实例独立，总量会超；Redis 集中式准但增加延迟。常见做法是「本地预取 + 定期同步」，在精度和性能之间折中。

限流键的选择也很重要：按用户、租户、IP、模型还是全局。选错了，要么误伤，要么形同虚设。

限流的目标不是「拒绝请求」，而是「让系统在过载时保持可预测」。

4. 准入控制怎么做？队列满了就拒绝为什么不对？

「队列满了就拒绝」的问题是把「排队长度」当成了「系统是否过载」。队列没满不代表能在可接受时间内完成，队列满了也不代表所有请求都该拒绝。

更合理的判据是「预计完成时间」：根据当前有效吞吐率和队列中已有工作量，估算新请求大概要等多久。如果超过 SLA，就拒绝或降级，而不是等它排到队尾。

有效吞吐率要动态估算，考虑上游限流、模型降级、工具耗时，不能用一个固定值。

拒绝时要给出可行动的信息：预计等待时间、建议重试时间、替代方案（比如用轻量模型、稍后再试）。只返回 429 而不给 Retry-After，客户端会立刻重试，反而加剧问题。

还要区分请求优先级：交互式请求优先于后台任务，付费租户可以有保留容量，但不能无限挤占。

5. 队列怎么设计？优先级、老化和队头阻塞怎么处理？

首先，队列必须有界。无限队列会把内存打满，而且让延迟无限增长；有界队列配合准入控制，才能保持可预测。

其次，要有优先级，但要防止低优先级饿死。常见做法是优先级 + 老化：等待越久，优先级逐渐提升。

第三，多租户场景要做公平调度。简单的 FIFO 会被大租户占满，可以用 WFQ（加权公平队列）或虚拟时间：每个租户维护一个虚拟时间，出队时选虚拟时间最小的，保证小租户也有机会。

第四，防队头阻塞。一个大任务卡住，不应该挡住后面的小任务。可以按任务类型分队列，或者把长任务异步化，不要让交互请求和后台任务挤在一起。

第五，队列要有可观测性：长度、等待时间、超时率、各租户占用。否则出了问题不知道是容量不足还是调度不公平。

6. 如何做削峰和请求合并？

三种常见手段。

第一，异步任务队列：把长任务从同步请求里拿出来，立即返回 Job ID，客户端轮询或订阅结果。这能解决连接占用和超

时问题。

第二，Singleflight：相同请求在同一时刻只发一次，其他请求等待同一个结果。适合幂等查询、相同 Prompt 的并发请求。注意不能合并有副作用的操作，也不能合并租户隔离要求不同的请求。

第三，微批（Micro-batching）：把短时间内的小请求合并成一批调用模型，降低单位成本。适合后台批量任务，不适合对延迟敏感的交互请求。

做合并时要考虑：缓存 key 怎么设计、结果怎么分发、失败怎么处理、不同租户能不能合并。合并错了会产生隐私和正确性问题。

削峰的本质是「把峰值摊平」，不是「把请求丢掉」。

7. Agent 场景的缓存怎么做?

我会分层做：精确缓存、语义缓存、工具结果缓存。

精确缓存按规范化后的 Prompt、模型、参数做 key，命中直接返回，最安全。

语义缓存把 Prompt 转成向量，相似度超过阈值就复用答案。它适合高频、答案稳定的问题，但最大的风险是假阳性：问题看起来相似，实际约束不同。所以要加安全校验：模型版本、租户、权限、时效、关键实体必须一致。

工具结果缓存要按工具、参数和时效做 key，只缓存只读或幂等结果。有副作用的结果不能随便缓存。

所有缓存都要有 TTL 和失效机制。知识更新、Prompt 更新、模型升级时，相关缓存要失效，否则会返回旧答案。

缓存 key 必须包含租户和权限范围，不能跨租户命中。

8. 网关的重试、熔断和降级怎么设计?

重试只在一层做，通常在网关。上层只判断错误类型，不重复发送。重试要满足：错误可重试、操作幂等、总预算允许。用指数退避加抖动，尊重 429 的 Retry-After。

熔断按上游、模型、接口维度统计错误率和延迟。超过阈值就打开，快速失败或切备用，过一段时间半开探测。熔断要能自动恢复，不能一直开着。

降级要有明确的等级：切小模型、关闭非关键工具、返回缓存结果、转人工、直接拒绝。降级策略要可配置、可灰度、可回滚。

跨模型 fallback 要注意：不同模型的能力和输出格式可能不同，不能假设无缝切换；切换后要做格式校验和效果监控。

最重要的是：重试、熔断、降级不能互相打架。比如熔断打开后还在重试，就失去了意义。

9. 长任务怎么做断点续跑和分布式调度?

核心是四件套：状态机、检查点、租约、幂等。

状态机定义任务生命周期和合法迁移；检查点保存执行到哪一步、上下文、工具结果；租约保证同一时刻只有一个 Worker 处理同一个任务；幂等保证重试不会产生重复副作用。

Worker 定期心跳续租。如果租约过期，Reaper 把任务回收，重新分配给其他 Worker。恢复时先读检查点，再核对工具执行状态，决定重跑、跳过还是转人工。

消息队列用来解耦和削峰，但不能完全依赖 MQ 的顺序和「只投递一次」。要用 Outbox Pattern 保证「业务状态变更」和「消息发送」的原子性，消费端做幂等。

检查点的粒度要权衡：太细成本高，太粗重跑多。通常在关键步骤和工具调用前后打点。

这套机制的目标是：进程可以崩、机器可以挂，但任务最终能完成，且不会重复产生副作用。

10. Agent 的优雅取消怎么做?

取消要贯穿四层。

第一层，交互层：客户端断开或用户点取消时，服务端要能收到信号，而不是继续跑。

第二层，循环层：Agent 的每一步检查取消令牌，在安全点退出，而不是强行杀进程。

第三层，上游调用：取消要传到模型请求和工具调用，释放连接和配额。很多系统只取消了自己的循环，上游还在烧 Token。

第四层，状态层：取消后要保存检查点，标记为 CANCELLED，已经产生的副作用要记录，必要时补偿。

取消还要幂等：重复取消不报错；取消后不能再继续执行；正在执行的工具调用要等它返回或超时，不能留下不确定状态。

优雅取消的价值是节省成本、避免脏状态，并让用户觉得系统「听话」。

11. 如何做压测和容量验证?

压测要模拟真实流量，而不是只打固定 QPS。

第一，构造有分布的请求：不同长度、不同工具、不同模型、不同租户，包含长尾任务。

第二，分层压测：先压网关，再压 Agent 运行时，再压上游模型（用 mock 或配额模拟），找出瓶颈在哪。

第三，观察关键指标：QPS、P95/P99 延迟、错误率、队列长度、连接数、Token 消耗、上游 429 比例、降级触发次数。

第四，做故障压测：模拟模型超时、限流、工具失败、实例宕机，验证熔断、重试和恢复。

第五，根据结果反推容量和降级阈值，不要把压测当成一次性的上线前动作。

压测的结论要落到配置：限流阈值、队列长度、超时时间、熔断阈值，都要有依据。

12. 线上应该重点看哪些指标?

我会分四组。

流量与容量：QPS、并发数、队列长度、等待时间、拒绝率、租户占用。

延迟：TTFT、P50/P95/P99、各阶段耗时（网关、检索、模型、工具）、超时率。

质量与成本：任务成功率、重试率、降级率、Token 消耗、单任务成本、缓存命中率。

稳定性：错误率、上游 429/5xx、熔断次数、租约过期、检查点恢复次数、取消成功率。

Agent 还要加一组业务指标：人工修改率、转人工率、采纳率。这些比纯技术指标更能说明系统有没有价值。

指标要能按模型版本、Prompt 版本、租户、任务类型下钻，否则出了问题只能看总量。

高频追问

- Little's Law 在什么情况下不适用?
- 本地限流和 Redis 限流怎么选?
- 语义缓存的假阳性怎么检测?
- 租约过期后，任务应该立刻被抢走吗?
- 取消时正在执行的不可逆工具调用怎么办?
- 上游模型限流时，应该排队还是直接降级?